Name: Aryan

Roll No: AE-1254

Course: B.Sc. (H) Computer Science

Semester: 8th

DSC- INFORMATION SECURITY

PRACTICAL

Q1. Demonstrate the use of Network tools: ping, ipconfig, ifconfig, tracert, arp, netstat, whois.

i. **ping:** ping google.com

```
C:\Users\admin>ping google.com

Pinging google.com [142.250.182.46] with 32 bytes of data:
Reply from 142.250.182.46: bytes=32 time=7ms TTL=118
Reply from 142.250.182.46: bytes=32 time=7ms TTL=118
Reply from 142.250.182.46: bytes=32 time=8ms TTL=118
Reply from 142.250.182.46: bytes=32 time=25ms TTL=118

Ping statistics for 142.250.182.46:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 7ms, Maximum = 25ms, Average = 11ms
```

ii. **ipconfig:** ipconfig

```
C:\Users\admin>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
```

iii. **ifconfig:** ifconfig

```
vineet@DESKTOP-BEJLPT6:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.22.26.153  netmask 255.255.240.0  broadcast 172.22.31.255
        inet6 fe80::215:5dff:fe88:e977  prefixlen 64  scopeid 0x20<link>
        ether 00:15:5d:88:e9:77  txqueuelen 1000  (Ethernet)
        RX packets 153  bytes 359633 (359.6 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 79  bytes 7916 (7.9 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 20  bytes 2249 (2.2 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 20  bytes 2249 (2.2 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

iv. **tracert:** tracert google.com

```
C:\Users\admin>tracert google.com

Tracing route to google.com [142.250.182.46]
over a maximum of 30 hops:

  1     5 ms     2 ms     1 ms  192.168.0.1
  2     3 ms     3 ms     6 ms  |
```

v. **arp:** arp –a

```
C:\Users\admin>arp -a

Interface: 192.168.2.103 --- 0xe
  Internet Address      Physical Address      Type
  192.168.2.1           04-95-e6-a3-37-c8     dynamic
  192.168.2.255         ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

vi. **netstat:** netstat –an

```
C:\Users\admin>netstat -an

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    0.0.0.0:135            0.0.0.0:0              LISTENING
  TCP    0.0.0.0:445            0.0.0.0:0              LISTENING
  TCP    0.0.0.0:5040           0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49664          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49665          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49666          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49667          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49668          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49670          0.0.0.0:0              LISTENING
  TCP    192.168.0.100:139      0.0.0.0:0              LISTENING
  TCP    192.168.0.100:6574     52.123.129.254:443     ESTABLISHED
  TCP    192.168.0.100:6575     23.50.12.8:443         ESTABLISHED
```

vii. **whois:** whois google.com

```
Server:  UnKnown
Address:  192.168.26.1

Non-authoritative answer:
Name:    google.com
Addresses:  2404:6800:4002:819::200e
          142.250.182.46
```

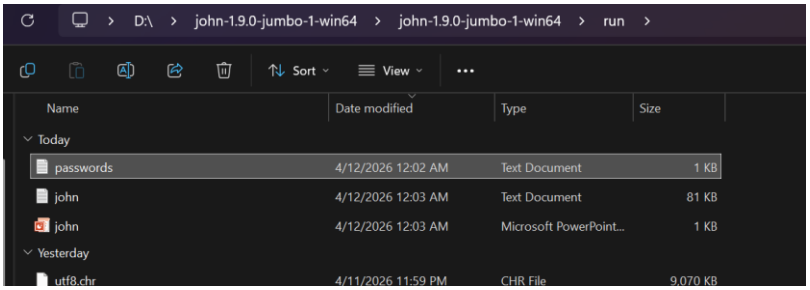
Q2. Use of Password cracking tools: John the Ripper, Ophcrack. Verify the strength of passwords using these tools.

Steps:

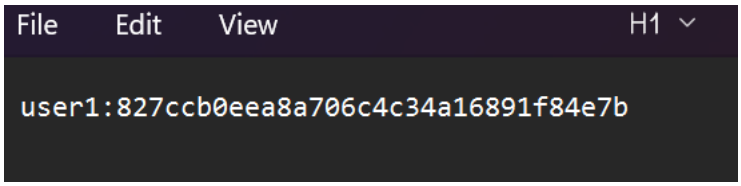
John the Ripper

Step 1: Download John the Ripper

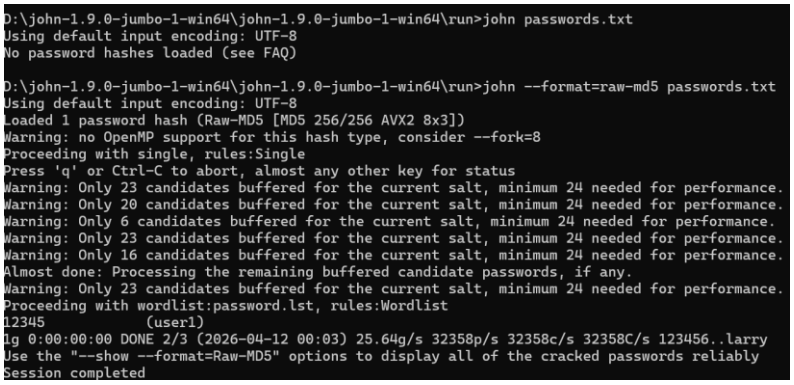
Step 2: Open Command Prompt in John Folder



Step 3: Create Password File

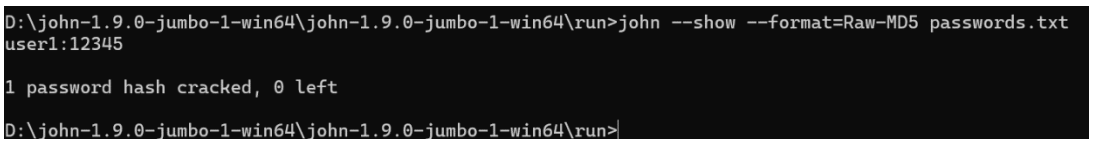


Step 4: Start Cracking Passwords



```
D:\john-1.9.0-jumbo-1-win64\john-1.9.0-jumbo-1-win64\run>john passwords.txt
Using default input encoding: UTF-8
No password hashes loaded (see FAQ)

D:\john-1.9.0-jumbo-1-win64\john-1.9.0-jumbo-1-win64\run>john --format=raw-md5 passwords.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=8
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 23 candidates buffered for the current salt, minimum 24 needed for performance.
Warning: Only 20 candidates buffered for the current salt, minimum 24 needed for performance.
Warning: Only 6 candidates buffered for the current salt, minimum 24 needed for performance.
Warning: Only 23 candidates buffered for the current salt, minimum 24 needed for performance.
Warning: Only 16 candidates buffered for the current salt, minimum 24 needed for performance.
Almost done: Processing the remaining buffered candidate passwords, if any.
Warning: Only 23 candidates buffered for the current salt, minimum 24 needed for performance.
Proceeding with wordlist:password.lst, rules:Wordlist
12345            (user1)
1g 0:00:00:00 DONE 2/3 (2026-04-12 00:03) 25.64g/s 32358p/s 32358c/s 32358C/s 123456..larry
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed
```

Step 5: View Cracked Passwords



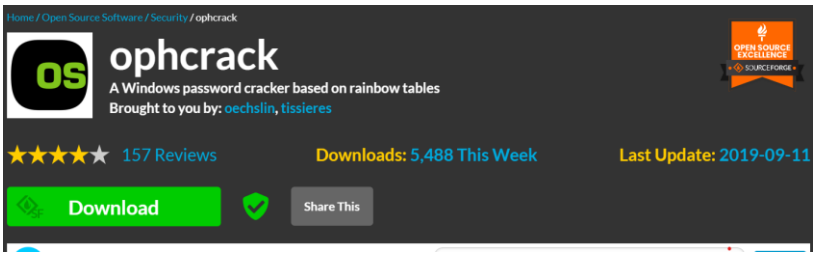
```
D:\john-1.9.0-jumbo-1-win64\john-1.9.0-jumbo-1-win64\run>john --show --format=Raw-MD5 passwords.txt
user1:12345

1 password hash cracked, 0 left

D:\john-1.9.0-jumbo-1-win64\john-1.9.0-jumbo-1-win64\run>
```

Step 6: Analyze Strength

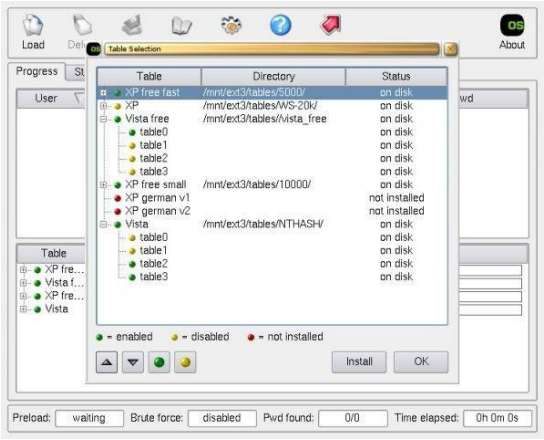
- 12345, password → cracked instantly
- hello123 → cracked quickly
- Strong@123 → harder to crack

OPHCRACK

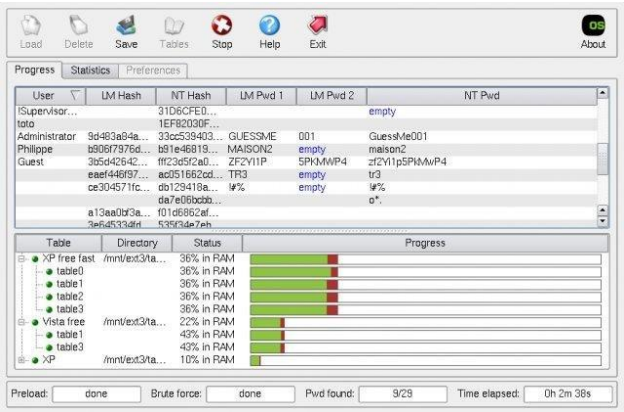
Step 1: Download Ophcrack



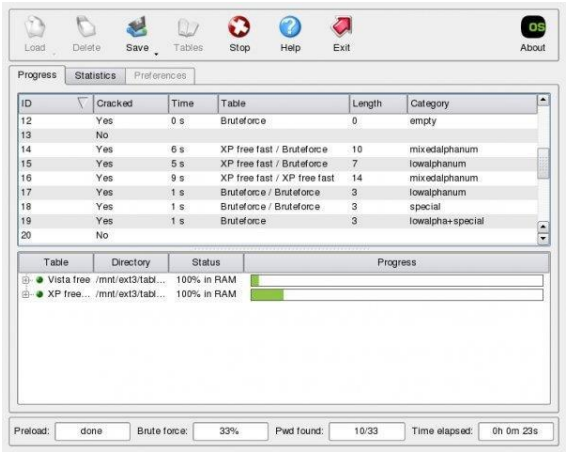
Step 2: Load Password Hashes (Local SAM)



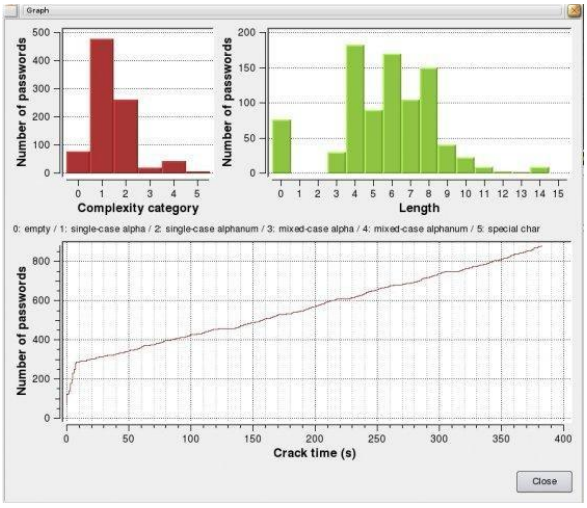
Step 3: Install Rainbow Tables



Step 4: Verify Rainbow Tables



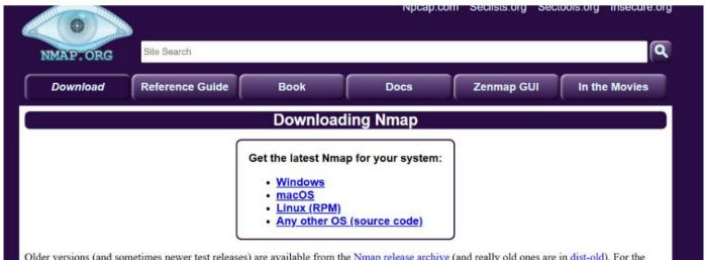
Step 5: Start Cracking + View Results

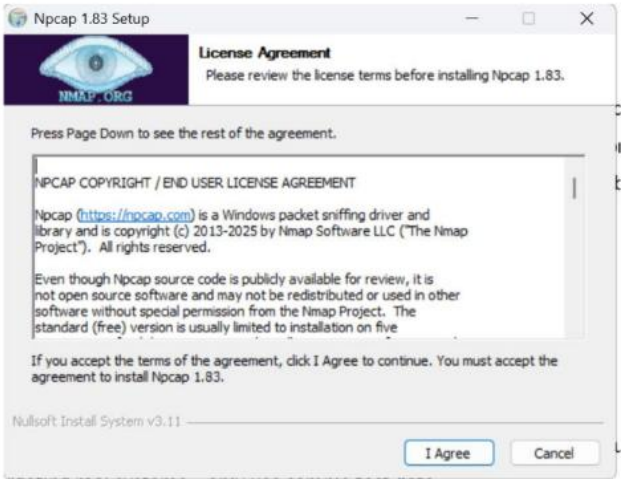


Q3. Use nmap/zenmap to analyze a remote machine.

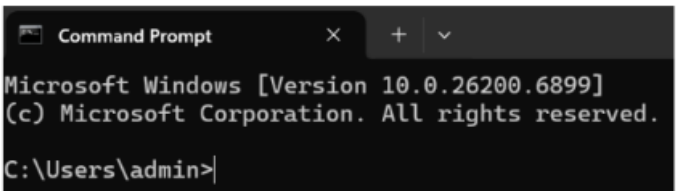
Steps:

Step 1 – Install Nmap



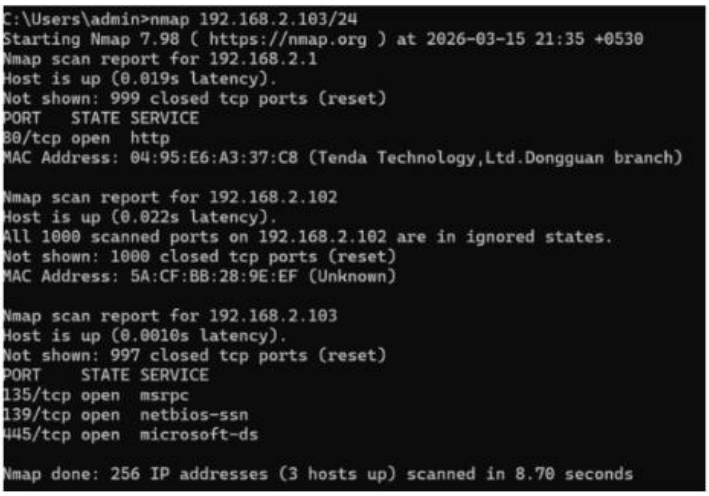


Step 2 – Open Command Prompt



Step 3 – Check Your Local IP Range

Step 4 – Scan the Local Network



Step 5 – Scan a Specific Host

```
C:\Users\admin>nmap 192.168.2.103
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-15 21:37 +0530
Nmap scan report for 192.168.2.103
Host is up (0.0010s latency).
Not shown: 997 closed tcp ports (reset)
PORT    STATE SERVICE
135/tcp open  msrpc
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds

Nmap done: 1 IP address (1 host up) scanned in 0.86 seconds
```

Step 6 – Perform Service Detection

```
C:\Users\admin>nmap -sV 192.168.2.103
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-15 21:38 +0530
Nmap scan report for 192.168.2.103
Host is up (0.00021s latency).
Not shown: 997 closed tcp ports (reset)
PORT    STATE SERVICE       VERSION
135/tcp open  msrpc         Microsoft Windows RPC
139/tcp open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp open  microsoft-ds?
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 7.66 seconds
```

Step 7 – Use NSE Scripts

```
C:\Users\admin>nmap --script=default 192.168.2.103
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-15 21:41 +0530
Nmap scan report for 192.168.2.103
Host is up (0.0010s latency).
Not shown: 997 closed tcp ports (reset)
PORT    STATE SERVICE
135/tcp open  msrpc
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds

Host script results:
| smb2-time:
|   date: 2026-03-15T16:11:37
|_  start_date: N/A
| smb2-security-mode:
|   3.1.1:
|_    Message signing enabled but not required

Nmap done: 1 IP address (1 host up) scanned in 17.50 seconds
```

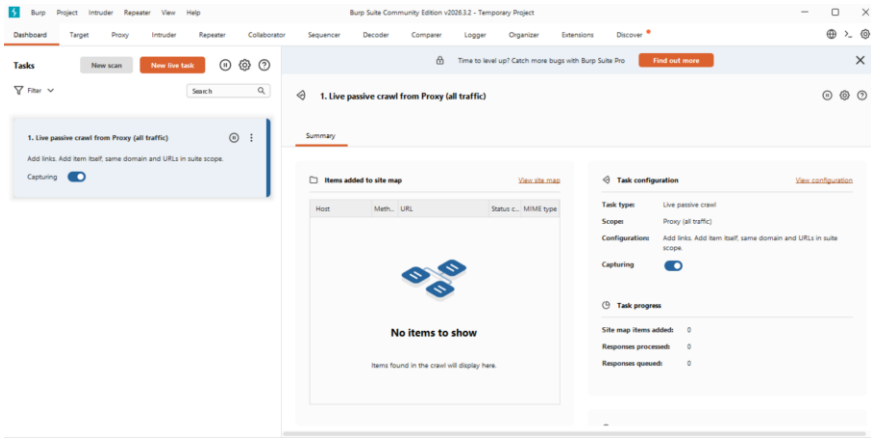
Result: The local network was successfully scanned using Nmap, and additional host/service information was obtained using Nmap Scripting Engine.

Q4. Use Burp proxy to capture and modify the message.

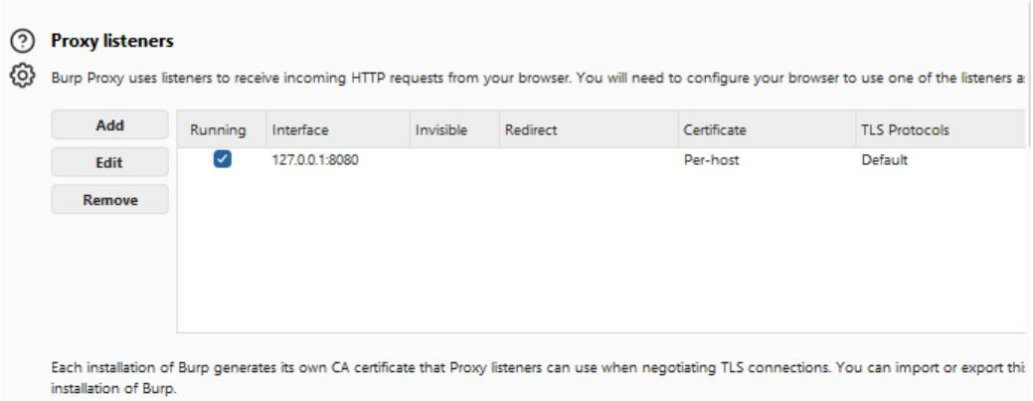
Steps:

Step 1: Download and Install Burp Suite

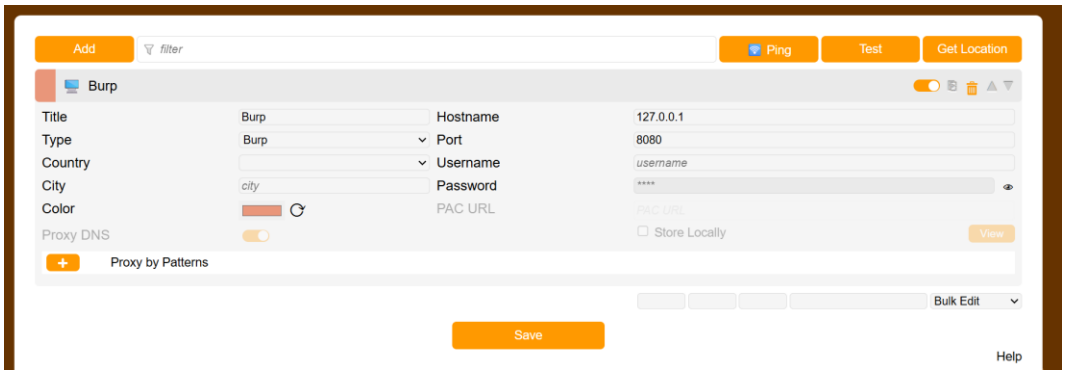
Step 2: Start Burp Suite



Step 3: Check Proxy Settings in Burp



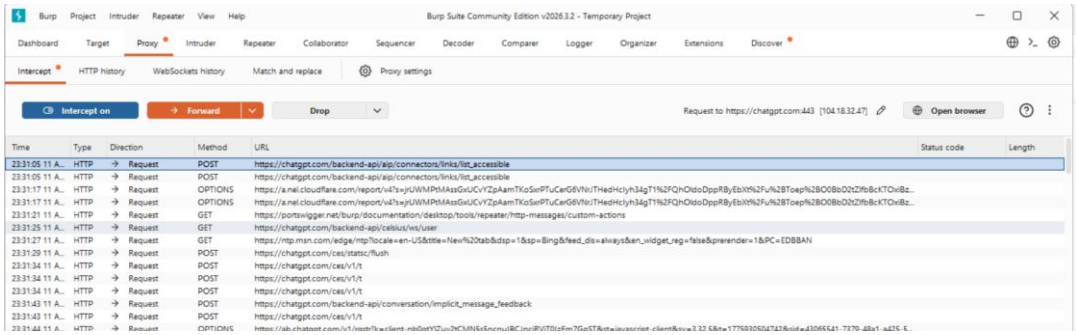
Step 4: Configure Browser Proxy



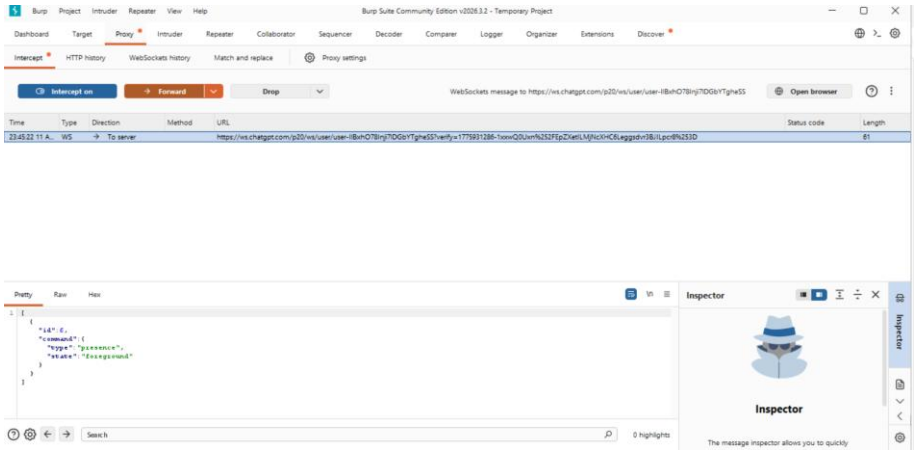
Step 5: Install Burp Certificate (for HTTPS)

Step 6: Turn ON Intercept in Burp

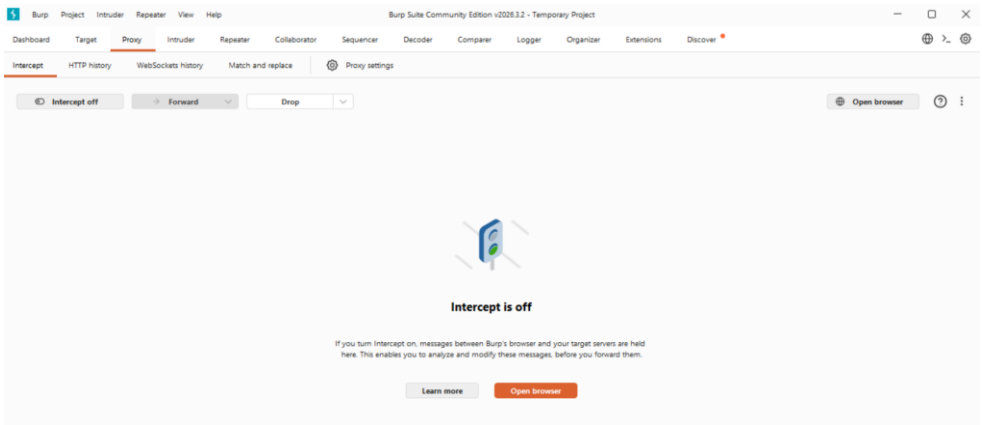
Step 7: Capture Request



Step 8: Modify the Message and Forward the Request



Step 9: Turn OFF Intercept



Q5. Implement caesar cipher substitution operation.

Code:

```
def caesar_encrypt(text, shift):
    result = ""

    for char in text:
        if char.isalpha():
            # Preserve case
            base = ord('A') if char.isupper() else ord('a')
            new_char = chr((ord(char) - base + shift) % 26 +
base)
            result += new_char
        else:
            result += char
    return result
#  Input
plaintext = input("Enter plaintext: ")
shift = int(input("Enter shift value: "))
cipher = caesar_encrypt(plaintext, shift)
print("Ciphertext:", cipher)
```

Output:

```
Enter plaintext: Information Security
Enter shift value: 3
Ciphertext: Lqirupdwlrq Vhfxulwb
```

Q6. Implement monoalphabetic and polyalphabetic cipher substitution operation.

Code:

```
import random
import string
# ---------------- MONOALPHABETIC ----------------
def generate_mono_key():
    alphabet = list(string.ascii_uppercase)
    shuffled = alphabet[:]
    random.shuffle(shuffled)
    return dict(zip(alphabet, shuffled))
def mono_encrypt(text, key):
    text = text.upper()
    result = ""
    for char in text:
        if char in key:
            result += key[char]
        else:
            result += char
    return result
# ---------------- POLYALPHABETIC (VIGENERE) ----------------
def vigenere_encrypt(text, key):
    text = text.upper().replace(" ", "")
    key = key.upper()
    result = ""
    key_index = 0
```

```
    for char in text:
        if char.isalpha():
            shift = ord(key[key_index % len(key)]) - ord('A')
            encrypted = (ord(char) - ord('A') + shift) % 26
            result += chr(encrypted + ord('A'))
            key_index += 1
        else:
            result += char
    return result
# ---------------- MAIN MENU ----------------
print("1. Monoalphabetic Cipher")
print("2. Polyalphabetic Cipher (Vigenere)")
choice = int(input("Enter your choice: "))
plaintext = input("Enter plaintext: ")
if choice == 1:
    key = generate_mono_key()
    print("\nGenerated Key Mapping:")
    for k, v in key.items():
        print(f"{k}->{v}", end="  ")
    cipher = mono_encrypt(plaintext, key)
    print("\n\nCiphertext:", cipher)
elif choice == 2:
    key = input("Enter key: ")
    cipher = vigenere_encrypt(plaintext, key)
    print("Ciphertext:", cipher)
else:
    print("Invalid choice!")
```

Output:

```
1. Monoalphabetic Cipher
2. Polyalphabetic Cipher (Vigenere)
Enter your choice: 1
Enter plaintext: Information security

Generated Key Mapping:
A->M  B->W  C->F  D->T  E->E  F->D  G->X  H->Y  I->S  J->V  K->Q  L->K  M->L  N-
>B  O->U  P->P  Q->R  R->C  S->G  T->O  U->H  V->I  W->J  X->N  Y->A  Z->Z

Ciphertext: SBDUCLMOSUB GEFHCSOA

================== RESTART: C:/Users/admin/Documents/IS_Q6.py ==================
1. Monoalphabetic Cipher
2. Polyalphabetic Cipher (Vigenere)
Enter your choice: 2
Enter plaintext: Information security
Enter key: 2
Ciphertext: TYQZCXLETZYDPNFCTEJ
```

Q7. Implement playfair cipher substitution operation.

Code:

```
import random

import string

def generate_random_key():

    alphabet = list("ABCDEFGHIKLMNOPQRSTUVWXYZ")

    random.shuffle(alphabet)

    return "".join(alphabet)


def generate_matrix(key):

    matrix = [list(key[i:i+5]) for i in range(0, 25, 5)]

    return matrix


def find_position(matrix, char):

    for i in range(5):

        for j in range(5):

            if matrix[i][j] == char:

                return i, j

def prepare_text(text):

    text = text.upper().replace("J", "I")

    text = "".join(filter(str.isalpha, text))
```

```
    pairs = []
    i = 0
    while i < len(text):
        a = text[i]
        if i+1 < len(text):
            b = text[i+1]
            if a == b:
                pairs.append(a + 'X')
                i += 1
            else:
                pairs.append(a + b)
                i += 2
        else:
            pairs.append(a + 'X')
            i += 1
    return pairs

def encrypt_playfair(text, matrix):
    pairs = prepare_text(text)
    result = ""

    for pair in pairs:
        r1, c1 = find_position(matrix, pair[0])
        r2, c2 = find_position(matrix, pair[1])

        if r1 == r2:  # Same row
            result += matrix[r1][(c1 + 1) % 5]
```

```
            result += matrix[r2][(c2 + 1) % 5]
        elif c1 == c2:  # Same column
            result += matrix[(r1 + 1) % 5][c1]
            result += matrix[(r2 + 1) % 5][c2]
        else:  # Rectangle rule
            result += matrix[r1][c2]
            result += matrix[r2][c1]
    return result

# Generate key
key = generate_random_key()
matrix = generate_matrix(key)
# Generated key matrix
print("Generated Key Matrix:")
for row in matrix:
    print(" ".join(row))
plaintext = input("Enter plaintext: ")
cipher = encrypt_playfair(plaintext, matrix)
print("Ciphertext:", cipher)
```

Output:

```
Generated Key Matrix:
Y K R G L
Q S U A I
W O P H Z
E V X T N
M D F B C
Enter plaintext: Information Security
Ciphertext: ZCDPYFHBSZVINMPUANRE
>>>
```

Q8. Implement hill cipher substitution operation.

Code:

```
def char_to_num(c):
    return ord(c) - ord('A')

def num_to_char(n):
    return chr(n + ord('A'))

def prepare_text(text, n):
    text = text.upper()
    text = "".join(filter(str.isalpha, text))

    while len(text) % n != 0:
        text += 'X'
    return text

def hill_encrypt(text, key, n):
    text = prepare_text(text, n)
    result = ""

    for i in range(0, len(text), n):
        block = text[i:i+n]
        nums = [char_to_num(c) for c in block]

        cipher_nums = []
        for row in key:
            value = sum(row[j] * nums[j] for j in range(n)) % 26
            cipher_nums.append(value)
        result += "".join(num_to_char(num) for num in
cipher_nums)
```

```
    return result

# Input matrix size
n = int(input("Enter size of key matrix (n for n x n): "))

# Input key matrix
print(f"\nEnter the {n}x{n} key matrix row by row")
print("Example for n=2:")
print("3 3")
print("2 5\n")

key = []
for i in range(n):
    row = list(map(int, input(f"Enter row {i+1}: ").split()))

    if len(row) != n:
        print("Error: You must enter exactly", n, "values")
        exit()
    key.append(row)

# Input plaintext
plaintext = input("\nEnter plaintext: ")

cipher = hill_encrypt(plaintext, key, n)
print("\nCiphertext:", cipher)
```

Output:

```
Enter size of key matrix (n for n x n): 2

Enter the 2x2 key matrix row by row
Example for n=2:
3 3
2 5

Enter row 1: 2 3
Enter row 2: 4 5

Enter plaintext: Information Security

Ciphertext: DTAMSYFRGYCMOANJVXND
```

Q9. Implement rail fence cipher transposition operation.

Code:

```
def rail_fence_encrypt(text, key):
    text = text.replace(" ", "")
    rail = [[' ' for _ in range(len(text))] for _ in range(key)]
    direction_down = False
    row, col = 0, 0

    # Fill zigzag
    for char in text:
        rail[row][col] = char
        col += 1
        if row == 0 or row == key - 1:
            direction_down = not direction_down
        row += 1 if direction_down else -1
    # Print zigzag pattern
    print("\nZigzag Pattern:")
    for r in rail:
        print(" ".join(r))
    # Read row-wise
    result = ""
```

```
    for i in range(key):
        for j in range(len(text)):
            if rail[i][j] != ' ':
                result += rail[i][j]
    return result


# Input
plaintext = input("Enter plaintext: ")
key = int(input("Enter number of rails: "))
cipher = rail_fence_encrypt(plaintext, key)
print("\nCiphertext:", cipher)
```

Output:

```
Enter plaintext: Information security
Enter number of rails: 3

Zigzag Pattern:
I       r       i       e       i
  n   o   m   t   o   s   c   r   t
    f       a       n       u       y

Ciphertext: Irieinomtoscrtfanuy
```

Q10. Implement row transposition cipher transposition operation.

Code:

```
import math
import random


def generate_key(n):
    key = list(range(1, n+1))
    random.shuffle(key)
    return key
```

```python
def row_transposition_encrypt(text, key):
    text = text.replace(" ", "")
    n_cols = len(key)
    n_rows = math.ceil(len(text) / n_cols)

    # Fill matrix row-wise
    matrix = []
    index = 0
    for i in range(n_rows):
        row = []
        for j in range(n_cols):
            if index < len(text):
                row.append(text[index])
                index += 1
            else:
                row.append('X')
        matrix.append(row)

    # Print matrix
    print("\nMatrix:")
    for r in matrix:
        print(" ".join(r))

    # Generate column order
    order = sorted(list(enumerate(key)), key=lambda x: x[1])
    # Read columns in key order
    cipher = ""
```

```
    for idx, _ in order:
        for row in matrix:
            cipher += row[idx]
    return cipher

# Input
plaintext = input("Enter plaintext: ")
n_cols = int(input("Enter number of columns: "))
key = generate_key(n_cols)
print("Generated Key:", key)
cipher = row_transposition_encrypt(plaintext, key)
print("\nCiphertext:", cipher)
```

Output:

```
Enter plaintext: InformationSecurity
Enter number of columns: 4
Generated Key: [4, 3, 2, 1]

Matrix:
I n f o
r m a t
i o n S
e c u r
i t y X

Ciphertext: otSrXfanuynmoctIriei
```

Q11. Implement product cipher transposition operation.

Code:

```
import math
import random
# Generate random key
def generate_key(n):
    key = list(range(n))
```

```
    random.shuffle(key)
    return key
# First Transposition (Row-wise fill, column read)
def transposition(text, cols, key):
    rows = math.ceil(len(text) / cols)
    # Fill matrix
    matrix = []
    index = 0
    for i in range(rows):
        row = []
        for j in range(cols):
            if index < len(text):
                row.append(text[index])
                index += 1
            else:
                row.append('X')
        matrix.append(row)
    print("\nMatrix:")
    for r in matrix:
        print(" ".join(r))
    # Read using key order
    cipher = ""
    for k in sorted(range(len(key)), key=lambda x: key[x]):
        for r in range(rows):
            cipher += matrix[r][k]
    return cipher

# Input
```

```
plaintext = input("Enter plaintext: ").replace(" ", "")
cols = int(input("Enter number of columns: "))
# First key
key1 = generate_key(cols)
print("Key 1:", key1)
# First transposition
step1 = transposition(plaintext, cols, key1)
print("\nAfter First Transposition:", step1)
# Second key
key2 = generate_key(cols)
print("\nKey 2:", key2)
# Second transposition
final_cipher = transposition(step1, cols, key2)
print("\nFinal Ciphertext:", final_cipher)
```

Output:

```
Enter plaintext: Information Security
Enter number of columns: 4
Key 1: [1, 3, 2, 0]

Matrix:
I n f o
r m a t
i o n S
e c u r
i t y X

After First Transposition: otSrXIrieifanuynmoct

Key 2: [1, 0, 3, 2]

Matrix:
o t S r
X I r i
e i f a
n u y n
m o c t

Final Ciphertext: tIiuooXenmriantSrfyc
```